The Mathematics of Vector Search in RAG

Bridging the Gap Between Theory and Billion-Scale Engineering

---

Applied Mathematics & AI

May 7, 2026

The Big Picture: RAG at Billion-Scale

The Goal: Retrieve contextually relevant documents from a corpus of $N > 10^9$ to ground Large Language Models (LLMs).

The Pipeline:

1. **Embedding (Bi-Encoders):** Map text to vectors $x \in \mathbb{R}^D$.
2. **Quantization (PQ):** Compress x to reduce RAM footprint.
3. **Partitioning (IVF):** Voronoi-based space division to avoid $O(N)$ scans.
4. **Graph Routing (HNSW):** Sub-logarithmic navigation between partitions.

1. Deconstructing RAG: Where is the Math?

What Exactly is RAG?

RAG operates like a specialized factory. The vector algorithms (HNSW/IVF) act as incredibly fast **warehouse workers**, while the LLM (ChatGPT/Gemini) is the **expert**. The algorithms don't "think" – they just fetch the right files so the expert doesn't have to guess.

- **R (Retrieval):** *The Math Engine.* Pure geometry and approximation (ANN, IVF, PQ). Goal: Minimize L_2 distance, maximize recall in < 100ms.
- **A (Augmented):** *The Engineering.* Taking the top-k results from the Retrieval phase and appending them to the user's prompt as context.
- **G (Generation):** *The LLM.* Autoregressive token generation based **only** on the retrieved context to eliminate hallucinations.

2. Vector Spaces and Distances

Distance Metrics in $\mathbb{R}^D$

How do we mathematically define if two sentences are "similar"? We measure the distance between their coordinates in a vector space.

Given query q and document x in $\mathbb{R}^D$ (typically $D \in [384, 1536]$):

- **Euclidean Distance (L_2):** $d(q, x) = \sqrt{\sum_{i=1}^{D}(q_i - x_i)^2}$
- **Inner Product (IP):** $IP(q, x) = \langle q, x \rangle = \sum_{i=1}^{D} q_i x_i$
- **Cosine Similarity:** $S_C(q, x) = \frac{\langle q, x \rangle}{\|q\|_2 \|x\|_2}$

Note: If vectors are L_2-normalized (i.e., $\|x\| = 1$), then minimizing L_2 distance is mathematically equivalent to maximizing Inner Product.

The Curse of Dimensionality

In 2D space, points can be close or far, but in high-dimensional spaces (e.g., 768D), everything is practically the same distance apart, making exact search useless.

By the law of large numbers, the pairwise distances between random vectors in high-dimensional space concentrate around the mean:

$$\lim_{D \to \infty} \frac{\text{dist}_{\text{max}} - \text{dist}_{\text{min}}}{\text{dist}_{\text{min}}} = 0$$

Conclusion: Exact k-NN ($O(N \cdot D)$ complexity) fails at scale. We must use **ANN (Approximate Nearest Neighbors)** to trade a small loss in Recall for a logarithmic speedup.

3. Product Quantization (PQ) and ADC

Product Quantization (PQ): Compression

The core idea of PQ is slicing a long vector into chunks and replacing each chunk with an ID from a predefined dictionary.

1. **Decomposition:** Split $x \in \mathbb{R}^D$ into m orthogonal subspaces: $\mathbb{R}^D = \mathbb{R}^{D/m} \times \cdots \times \mathbb{R}^{D/m}$.
2. **K-Means Training:** Learn a codebook $\mathcal{C}_j$ of size k^* (usually 256) for each subspace j.
3. **Encoding:** Map sub-vector u_j to the nearest centroid: $q_j(u_j) = \arg\min_{c \in \mathcal{C}_j} \|u_j - c\|^2$.

PQ Memory and Distortion Math

Memory Footprint: A standard vector is $D \times 4$ bytes (float32). With PQ ($m = 8$, $k^* = 256$), we store 8 indices of 1 byte each. Compression ratio: $\frac{768 \times 4}{8} = 384\times$.

Quantization Error (MSE): PQ minimizes the distortion $\mathcal{E}$ assuming subspaces are statistically independent:

$$\mathcal{E} = \sum_{j=1}^{m} \mathbb{E}\left[\|u_j - q_j(u_j)\|^2\right]$$

Trade-off: Increasing m reduces $\mathcal{E}$ (better accuracy) but increases storage size.

ADC: Asymmetric Distance Computation

We avoid unpacking the data to search. Instead, we create a Lookup Table (LUT) for the query and simply sum the pre-computed values.

To find the squared L_2 distance between an uncompressed query y and a PQ-compressed vector x:

$$\hat{d}(y, x)^2 = \sum_{j=1}^{m} \|u_j(y) - c_{j,i_j(x)}\|^2$$

Complexity:

1. Build LUT: $O(m \times k^* \times \frac{D}{m}) = O(D \cdot k^*)$ operations (done once).
2. Search: $O(m)$ additions per vector in the database (zero multiplications).

4. Partitioning & Graphs: IVF + HNSW

IVF: Voronoi Partitioning

By grouping vectors into clusters (buckets), we restrict our search to only a small fraction (e.g., 1%) of the database.

We run K-Means on the entire dataset to find K coarse centroids (parameter `nlist`). This divides the space into Voronoi cells.

Search Parameters:

- `nlist`: Number of clusters (e.g., $4\sqrt{N}$).
- `nprobe`: Number of clusters to visit during search.

Complexity: Search space is reduced from N to approximately $N \cdot \frac{nprobe}{nlist}$.

HNSW: Hierarchical Navigable Small World

HNSW operates like a layered map of highways and local streets. The search jumps across the map on the highway layer, then drops down to local streets to find the exact target.

The IVF Synergy: In a billion-scale system, building an HNSW graph on ALL vectors consumes too much RAM. Instead, the HNSW graph is built **only between the IVF centroids (capitals)**.

- **Layer Assignment:** $l = \lfloor -\ln(U(0,1)) \cdot m_L \rfloor$.
- **Role:** HNSW routes the query to the best *nprobe* buckets in sub-milliseconds.

5. The Full System Lifecycle

The Timeline: How it all connects

We index the data and build the navigation structures before the user even asks a question. During inference, we traverse the pre-built graphs and use our LUTs.

Phase 1: Offline (Indexing)

1. **IVF:** Cluster vectors into buckets (centroids).
2. **HNSW:** Build a highway graph connecting only those centroids.
3. **PQ:** Compress the vectors inside the buckets into byte-codes.

Phase 2: Online (Querying)

1. **HNSW:** Query travels the highway to find the top 5 buckets.
2. **ADC:** Create the Lookup Table (LUT) for the query.
3. **PQ:** Quickly sum values from the LUT for vectors inside those 5 buckets.

Summary

Summary: The Billion-Scale Vector Stack

Algorithm	Mathematical Role	Engineering Benefit
Embeddings	$\phi : \text{Text} \to \mathbb{R}^D$	Captures semantic topology.
PQ	Subspace K-Means ($x \approx \sum q_i$)	$O(1)$ memory reduction (e.g., 64x).
ADC	Look-Up Table L_2 estimation	Replaces $O(D)$ FLOPS with $O(m)$ additions.
IVF	Voronoi Partitioning	Limits search space to $\frac{nprobe}{nlist}$.
HNSW	Greedy Routing in $O(\log N)$	Sub-millisecond cluster matching.
DiskANN	I/O Bounded Graph Pruning (α)	Bypasses RAM limits using NVMe SSDs.